

Final Presentation

NPU Challenge

Team Name : NPU IS ALL YOU NEED

Research Topic : Optimizing LLM in long-context scenario on Eagle N NPU

2026-08-21

Muny Kim, HyunTaek Choi, Taejun Lee, Boseong Kim

SungKyunKwan University, Korea

spacemuny@gmail.com



Overview

1. Reasearch Background
2. Baseline profiling
3. Quest
4. TTNN/TT-Metal Optimization
5. Experiment
6. Bug Fix
7. Conclusion

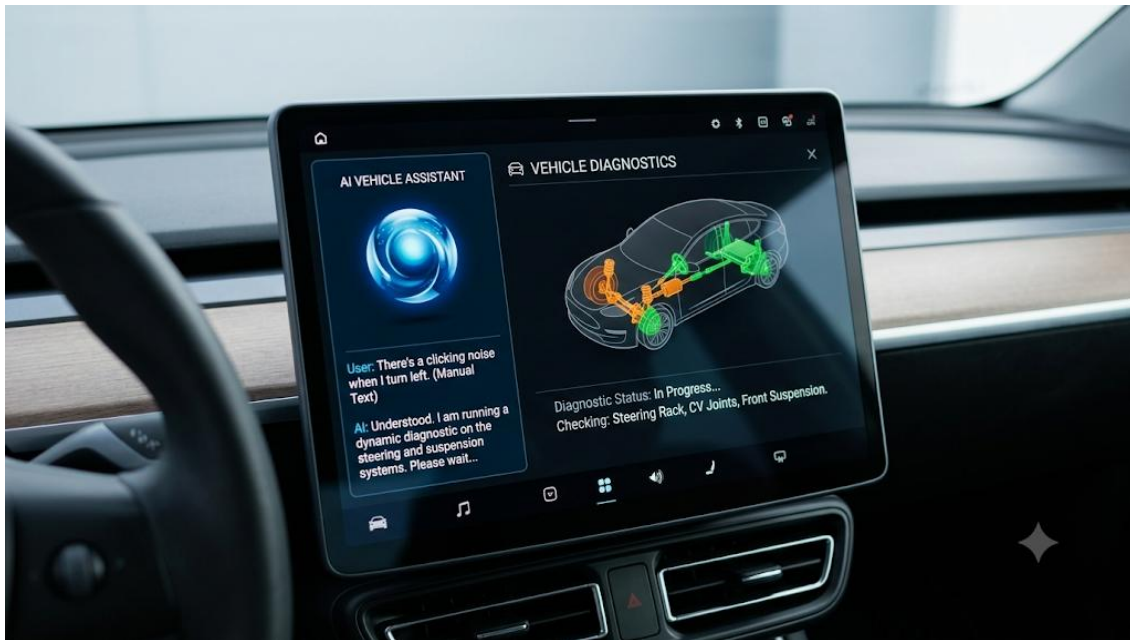
Appendix A. Kernel Optimization Rationale

Appendix B. Quest criticality

Research Background

Research Background

Why On-device LLM in cars?



- **The Rise of LLMs in Cars**

- LLMs enable natural, conversational interfaces for controlling vehicle hardware and diagnostics.

- **Why On-device LLM?**

- **Ultra-Low Latency:** Crucial for real-time driving safety.
- **Connectivity Independence:** Works seamlessly in tunnels and dead zones.
- **Data Privacy:** In-cabin data remains locally on the edge device.

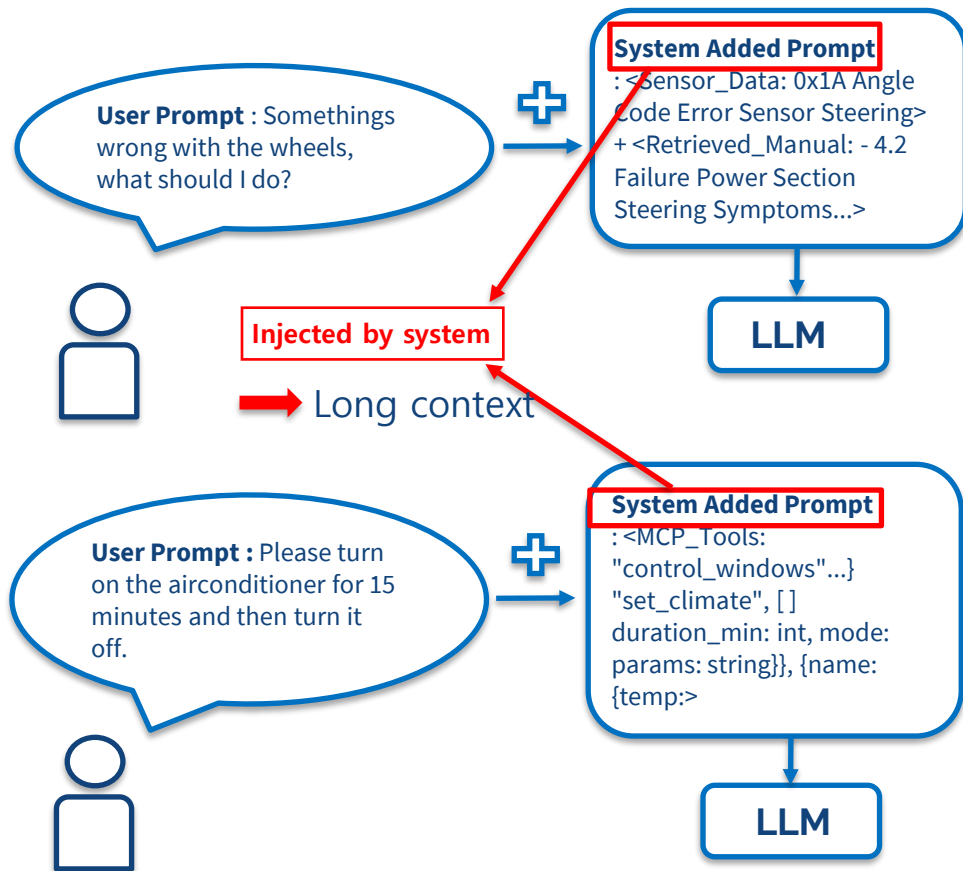
Research Background

Why long context scenario?

- **Why long context scenario?**

- Vehicle LLMs must interact with vast amounts of data injected by the system other than user input.
 - **RAG** : Large chunks of locally stored **vehicle manuals**, real time **diagnostic data** added to LLM input.
 - **MCP** (Model Context Protocol) : MCP **API function call manual** added to LLM input as system 'prompt'.

➡ **Need for 'Long context LLM optimization'**



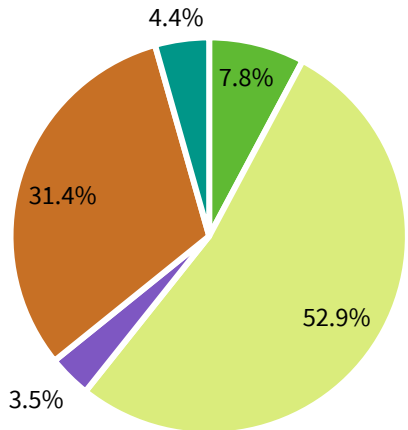
Baseline Profiling

- analyzing decode stage of baseline implementation

Baseline Profiling

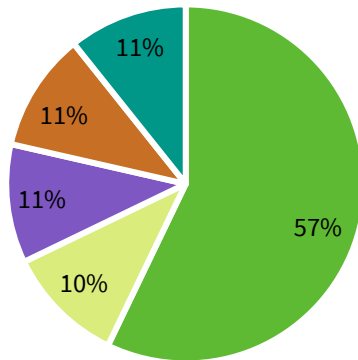
Runtime and DRAM Footprint, Bottleneck Identification

Decode-stage Runtime Breakdown



■ QKV projection ■ SDPA ■ Output projection ■ MLP(3 stage) ■ etc

Per-layer DRAM Footprint at 64K Context

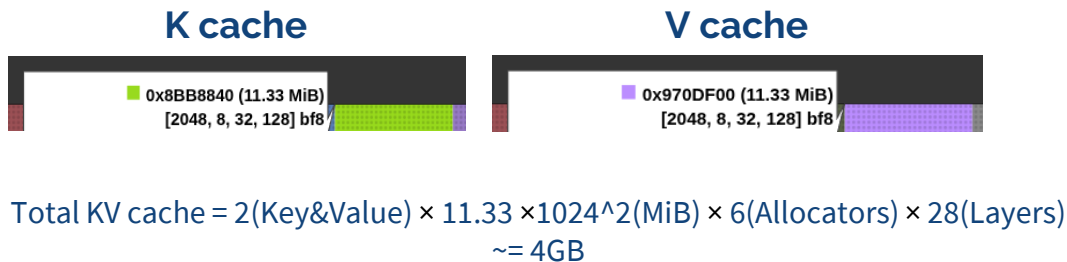
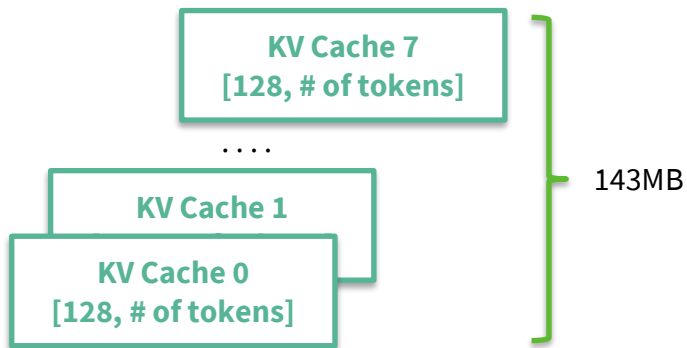


■ KV cache ■ MLP 1 weight ■ MLP 2 weight ■ MLP 3 weight ■ Others

- The most **time-consuming** layer : **SDPA**
- The most **memory consuming** layer : **SDPA (KV cache)**
- SDPA : $\text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$
 - Baseline SDPA optimization : Flash Attention / Paged KV cache
 - **Additional SDPA optimization available : Selective KV cache access methods -> experiment with (snapkv, flowkv, quest).**

Baseline Profiling

DRAM bandwidth utilization



Workload	DRAM Bandwidth
DRAM μ Bench	96 GB/s
SDPA Decode (Matmul part)	38.5 GB/s

- Total KV cache (64k context, 8 KV heads, 28 layers) : $2(\text{K\&V}) \times 65,536 (64\text{k}) \times 128 (\text{dim}) \times 1\text{B} (\text{BFP8}) \times 8 (\text{heads}) \times 28 (\text{layers}) = \mathbf{3.76 \text{ GB}}$
- BFP8 metadata overhead included (64B per 1024 elements) : $3.76 \text{ GB} \times 1.0625 \approx \mathbf{4 \text{ GB}}$
- Observation : SDPA is **memory bound** and **does not even fully utilize the DRAM bandwidth**.
- Optimization point : **Find ways (TTNN/TT-Metal optimizations) to at least fully utilize the DRAM bandwidth.**

Quest

Principle, our tt-nn implementation, results

Based on "Query-Aware Sparsity for Efficient Long-Context LLM Inference" — Tang et al., ICML 2024 (arXiv 2406.10774)

Quest

Principle : query-aware page sparsity – no eviction

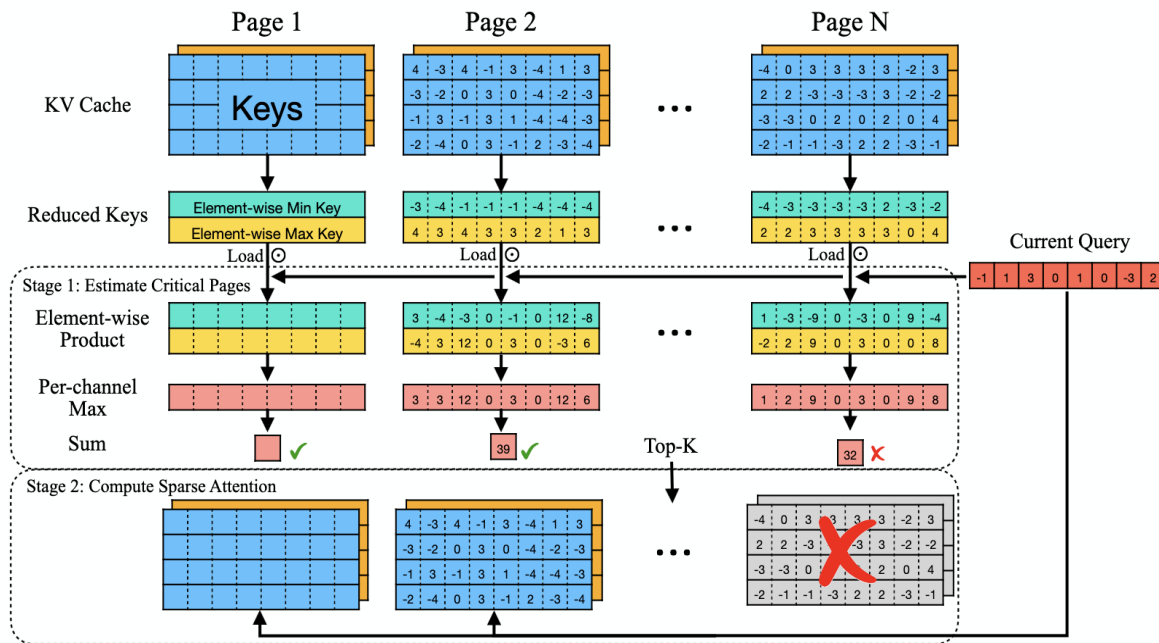


Fig. 5 two stages: min/max criticality → top-K pages.

- Quest reads only the pages important to the current query - no eviction

Stage ① : Score each page by criticality

- Min/Max-Key summary × query
- Appendix B.

Stage ② : Attend the top-K pages only

- the rest **skipped, not deleted**

Why Quest?

Over 90% of the KV is unused per query

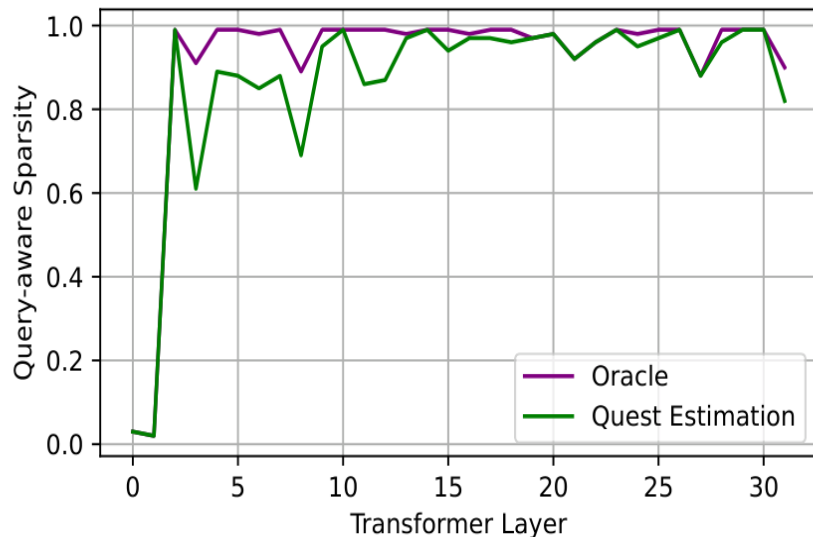


Fig. 3 over 90% of the KV can be skipped per query

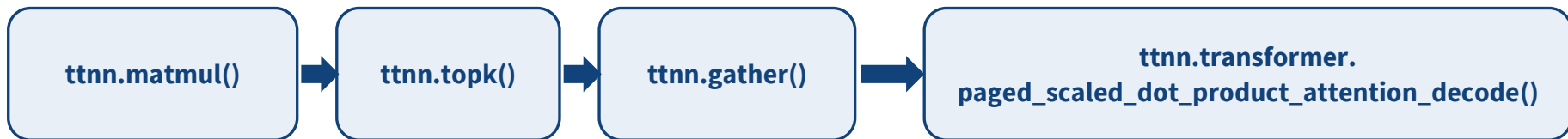
Sparsity (y-axis) = the share of the KV a query can skip
0.9 = 90% skippable

- **Every decode step re-reads the full KV** : at long context, KV read dominates decode time
 - **But each query needs only a few tokens** : over 90% of the KV is unused every step (Fig 3).
 - **Criticality is query-dependent** : the critical set changes every step → static eviction can't capture it.
- **Quest** : pick the query-critical pages each step, read only those, **keep the full KV** (no eviction).

Quest

Implementation : no custom kernel, existing tttn only

- **Original Quest:** custom CUDA kernels on **GPU**.
- We implemented Quest by composing **only built-in tttn functions** – zero kernel changes.
- **PREFILL (once)** : build a per-page **Min / Max - Key summary**
- **DECODE (every step)** : operation chain below



What each op does

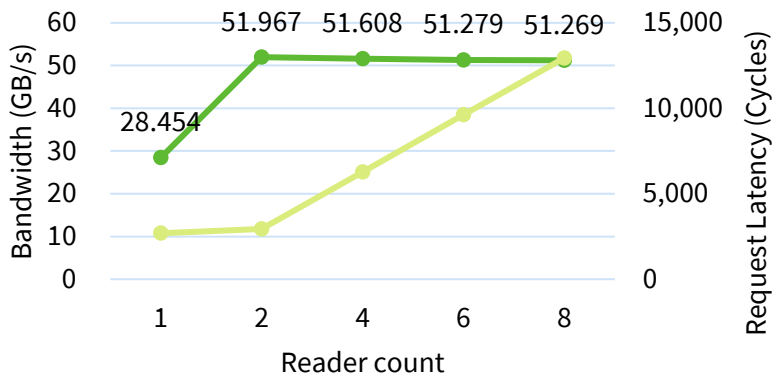
- **tttn.matmul()** : scores each page's criticality (query × the page's Min / Max-Key summary)
- **tttn.topk()** : selects the top-K (= budget) pages, skips the rest
- **tttn.gather()** : collects the selected pages into a compact page-table
- **tttn.transformer.sdpa.decode()** : runs paged attention on the selected pages only

TTNN/TT-Metal Optimization

Kernel Optimization

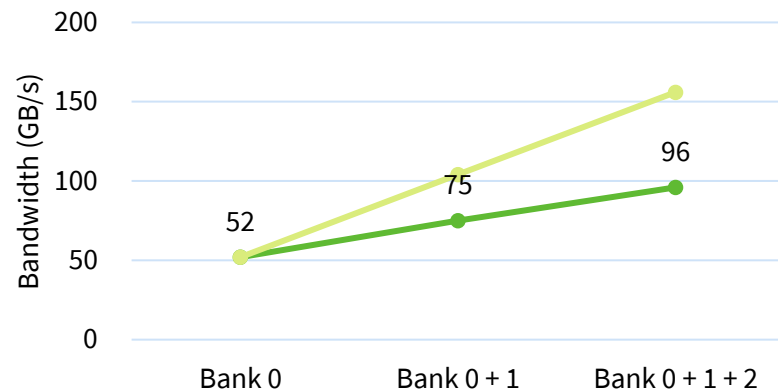
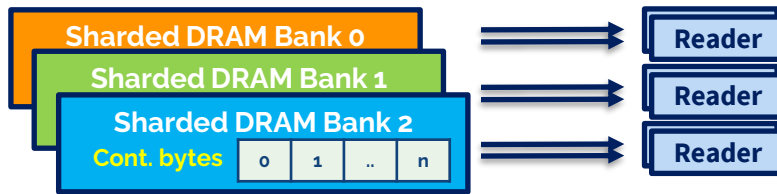
Measuring Memory Ceilings

Saturating 1 bank – N reader/s



● Bandwidth (GB/s) ● Request latency (Cycles)

Saturating 3 bank – 2 reader per bank



● Measured Bandwidth ● Ideal Bandwidth

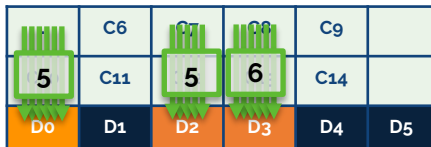
- A single bank saturates at **~52 GB/s** with one reader per endpoint.
- All three banks reach an aggregate read bandwidth of **~96 GB/s** using all six endpoints.

Kernel Optimization

Before Optimization

Scale dot product attention

1. Imbalance Reader-Endpoint matching & Few active endpoints



2. Unnecessary DRAM-DRAM data flow during concatenation

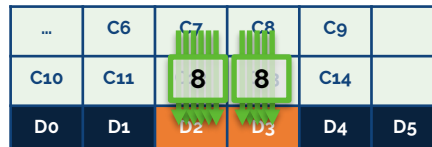


3. High Fixed Cost at Flash Attention due to low chunk size

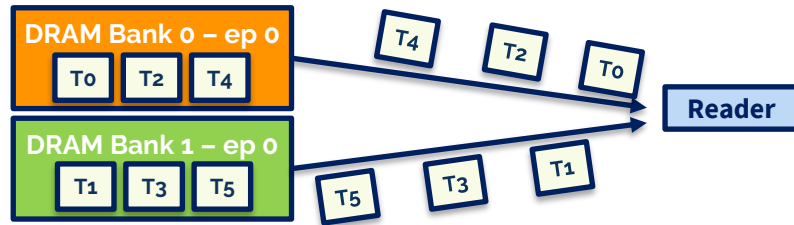


Feed Forward (including Wo & QKV matmul)

1. Imbalance Reader-Endpoint matching & Few active endpoints



2. Using continuous weight but interleaved at DRAM



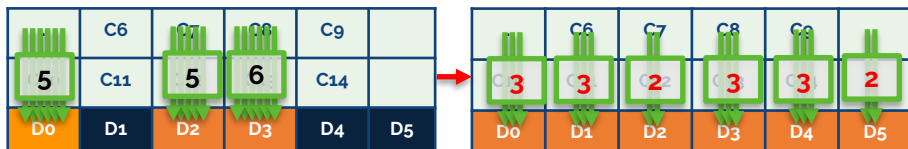
- The baseline did not fully exploit the memory and interconnect architecture, particularly DRAM-endpoint parallelism and efficient request granularity.

Kernel Optimization

After Optimization

Scale dot product attention

1. Balance Reader-Endpoint matching & Utilize all endpoints



2. Fuse concatenation operation

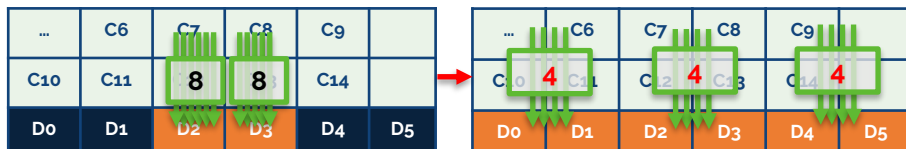


3. Double chunk size to reduce Fixed Cost

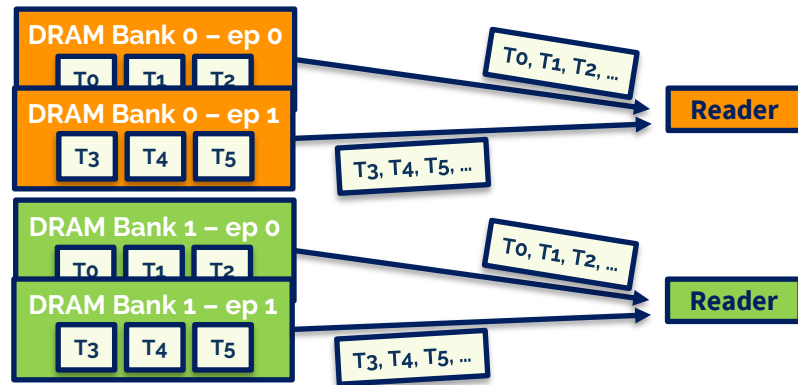


Feed Forward

1. Balance Reader-Endpoint matching & Utilize all endpoints



2. DRAM width-sharded weight layout with contiguous shard-local reads

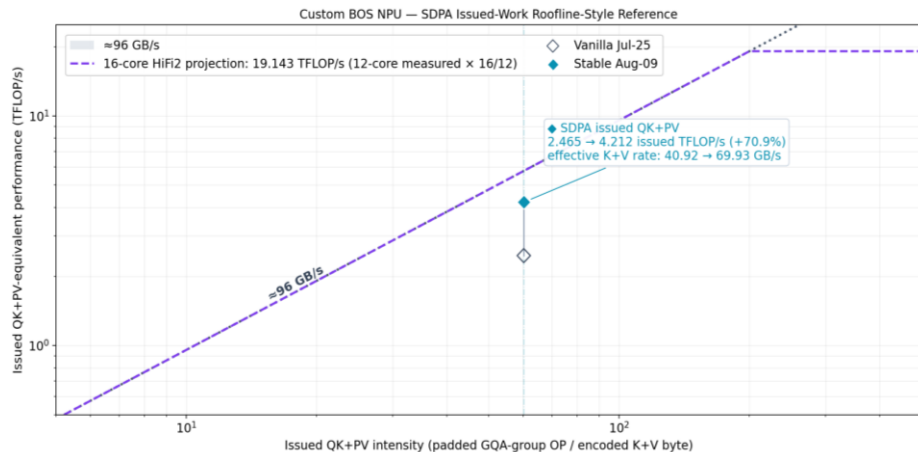


- We optimized the **SDPA** and **FeedForward** layers by **adopting memory-access patterns** closer to those of our high-bandwidth DRAM microbenchmark, while **eliminating redundant** computation and data movement.
- Maintaining a PCC of 0.9993** against the vanilla full-model logits, with the same top-1 prediction and complete top-5 overlap.

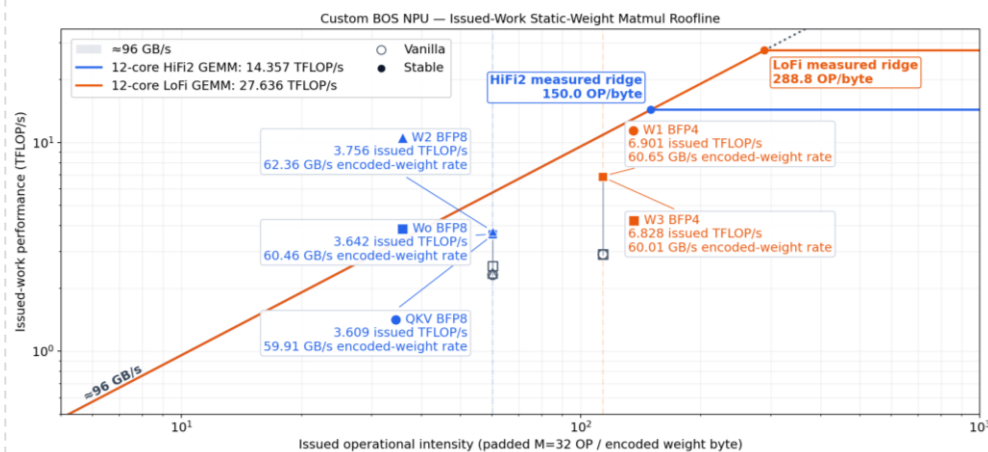
Kernel Optimization

Optimization Results

Scale Dot Product Attention



Feed Forward



Metrics	Vanilla	Optimized	Improvements
tokens/s	5.12	8.20	+60.16%

- We moved **SDPA** and **static-weight matmul** closer to the empirical roof by applying the memory-access principles identified through architecture characterization.
- End-to-end 64K decode throughput increased from **5.123 to 8.204 tokens/s** — a **60.15% improvement** over the actual vanilla path.

Experiment

Experiment

Setup

1 Long-Context Evaluation Benchmarks (Context Length: up to 64K)

- **NIAH (Needle in a Haystack)**
 - Evaluates whether KV can retrieve key information from anywhere in the context
- **RULER Retrieval**

Evaluate retrieval under more complex conditions using **Multi-Key, Multi-Value, and Multi-Query** tasks

Finds correct information among distractors and retrieves multiple targets
- **Variable Tracking**

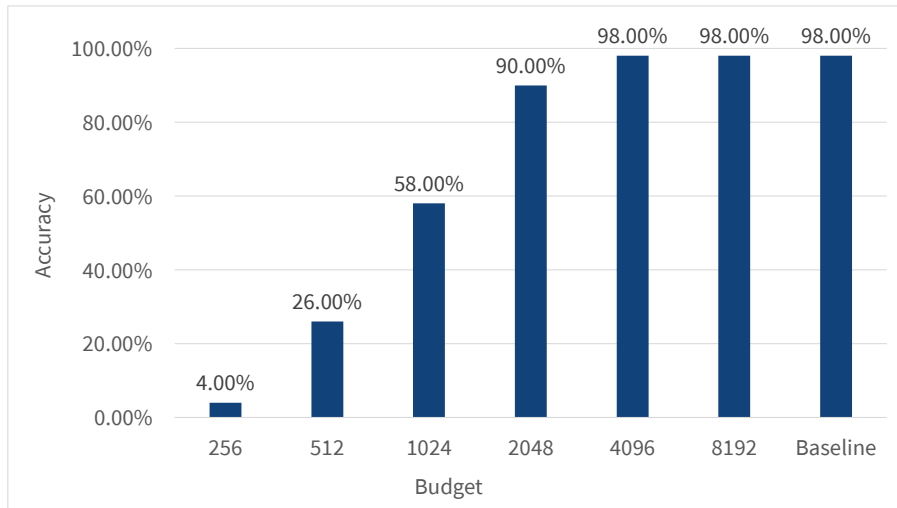
Evaluates whether the model can follow chained variable relationships across long contexts
- **# of samples: 50 (64k context length)**

2 Model

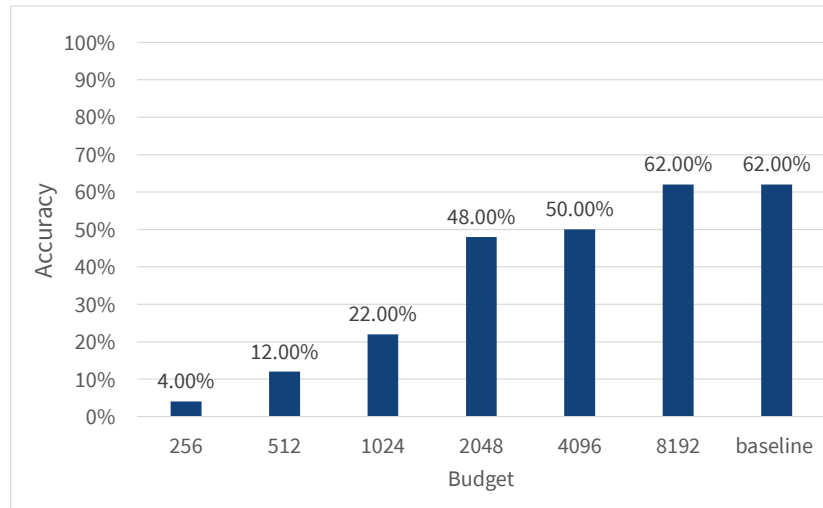
- **Model:** Llama 3.2 3B
- **DRAM size:** 12GB (3 banks, 4GB per banks)

Experiment

Benchmark Result: NIAH & RULER(Multi-key)



NIAH Result



Multi-Key Result (1 Target, 3 Distractors)

- **Budget Effect** : Retrieval accuracy increases as the KV budget increases
- **Multi-Key Difficulty**: Multi-Key requires a larger budget than NIAH to reach baseline accuracy

Experiment

Benchmark Result: RULER(Multi-Value, Multi-Query)



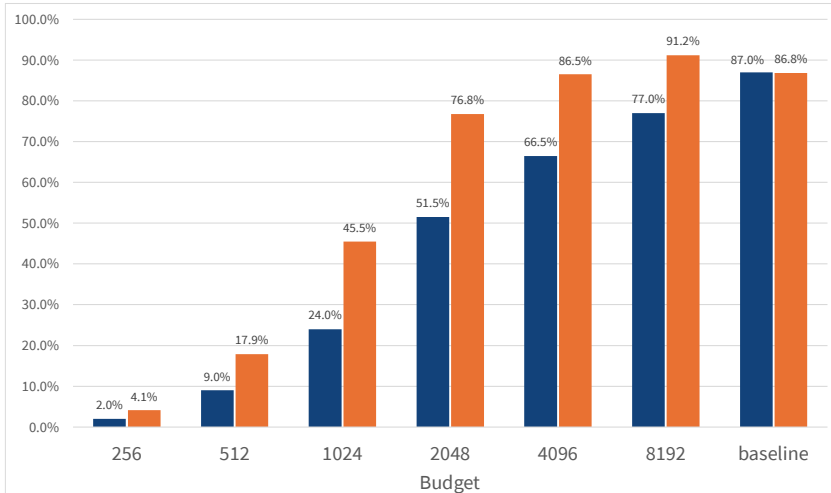
Precision



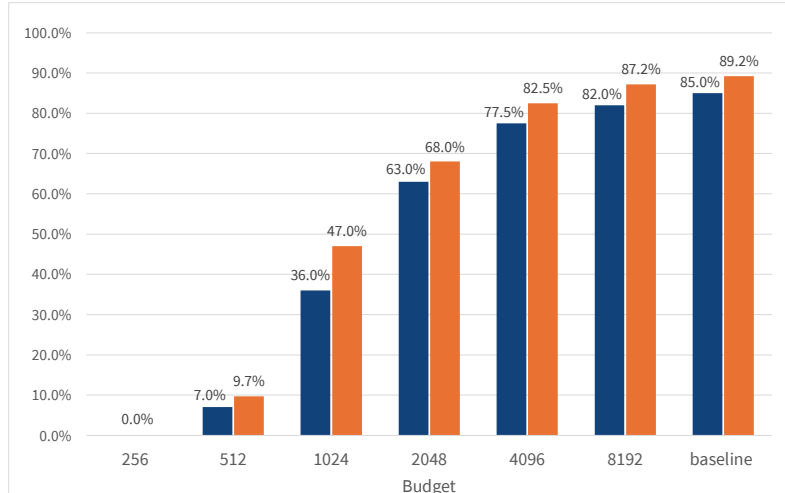
Score

Precision = Correct Targets / Total Outputs

Score = Correct Targets / Total Targets



Multi-Value (4 Target)



Multi-Query (4 Target)

- **Budget Effect** : Both Score and Precision improve as the KV budget increases
- **Multi-Target Difficulty**: Larger budgets help retrieve more targets while reducing incorrect outputs

Experiment

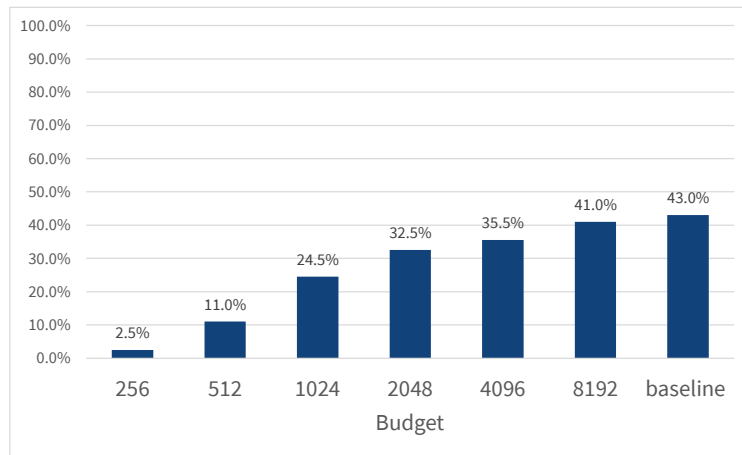
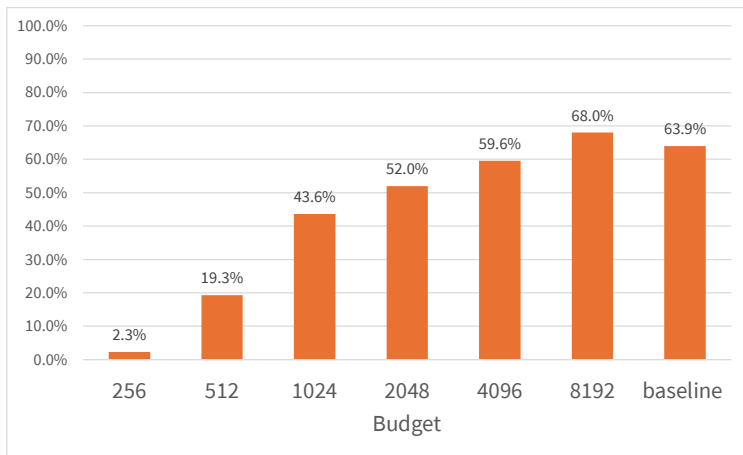
Benchmark Result: Variable Tracking

■ Precision

■ Score

Precision = Correct Targets / Total Outputs

Score = Correct Targets / Total Targets



VAR_A = 6806409

↓

VAR_B = VAR_A

↓

VAR_C = VAR_B

↓

VAR_D = VAR_C

+ Distractors

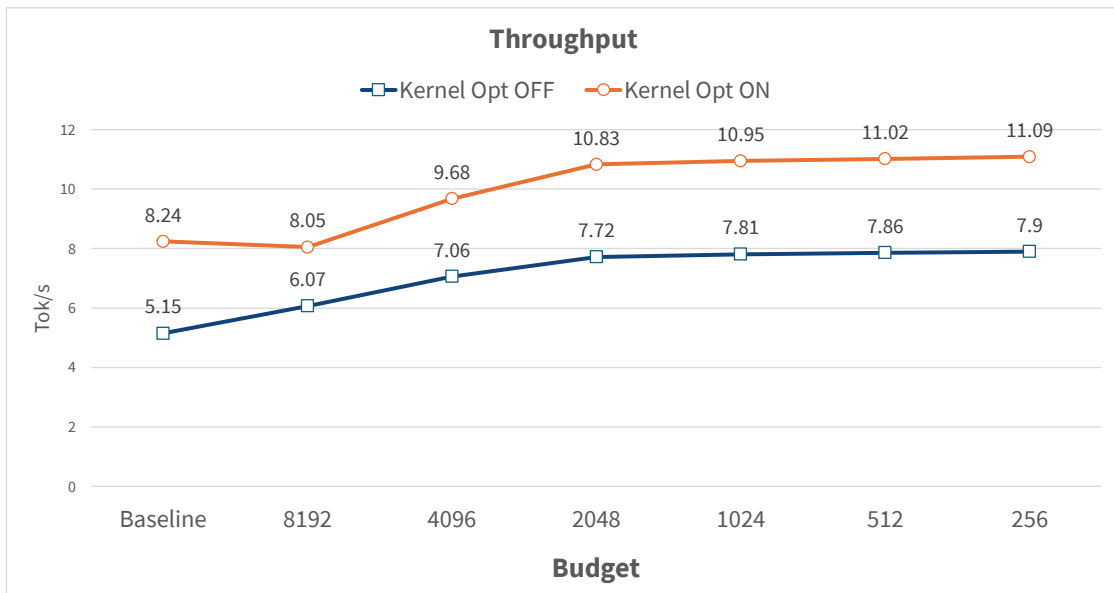
Trace a 4-step
variable chain

Variable tracking (4 Targets, 4 Distractors)

- **Target Coverage:** Score increases with the KV budget, reaching 41.0% at 8K, close to the 43.0% baseline
- **Output Quality:** 8K achieves 68.0% Precision, with fewer incorrect outputs

Experiment

Benchmark Result: Throughput



Throughput (Generate 512 tokens)

Budget	Score
Baseline	100
b8192	94.1
b4096	84.9
b2048	71.2
b1024	41
b512	14.3
b256	2.5

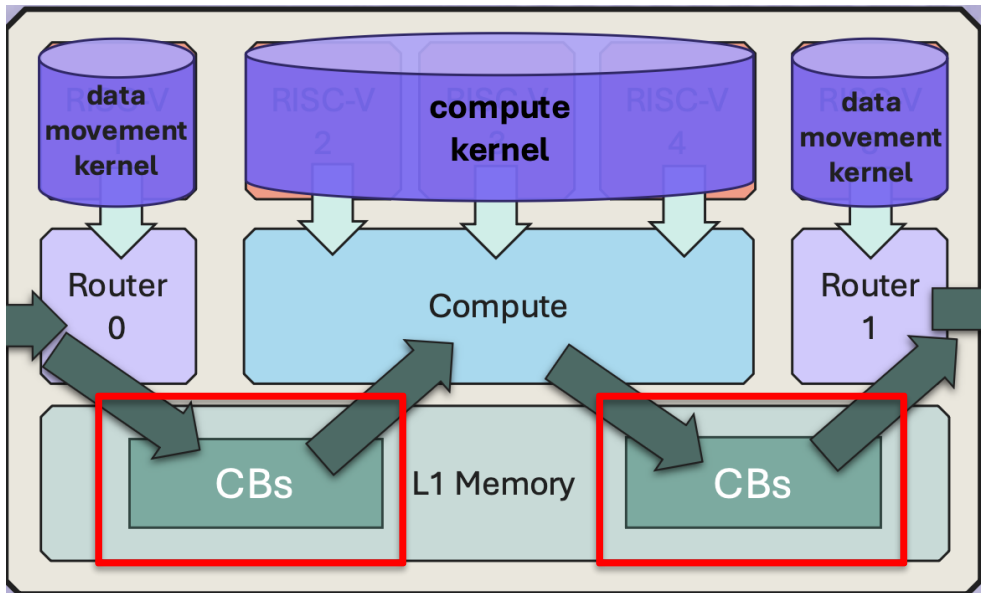
Overall Benchmark Score
(NIAH + RULER + Variable Tracking)

- **Gather Overhead:** 8K shows limited speedup due to increased gather overhead
- **Trade-off:** Smaller KV budgets increase throughput but reduce benchmark accuracy
- **Best Balance:** 4k-2k provide the best throughput-accuracy trade-off

Bug Fix

Bug Fix

SDPA hang : an intra-Tensix memory-ordering race condition

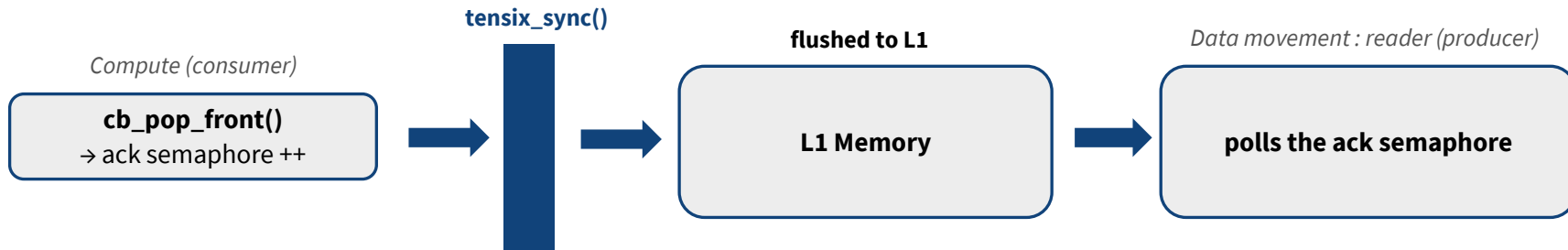


Tensix core — 3 kernels (1 compute + 2 data-movement) hand off through CBs in the shared L1 (Tenstorrent pdf)

- **Symptom** : at 8k+ context, SDPA hangs → **device locks up** (HW reset needed)
- **Cause** : 5 RISC-V cores hand off via **CBs in L1**; Blackhole **reorders / never makes visible** the **`cb_pop_front()`** ack → core waits forever
- **Why a race condition** : L1 **ordering not guaranteed**; worse at longer context = **race fingerprint**
- **Not our bug** : BOS's own code already treats it as a **hardware hang** ([BOS-ARCH-006] "llk hang" workaround), patched in matmul / conv / batch-norm — **SDPA was missed**.

Bug Fix

SDPA hang : an intra-Tensix memory-ordering race condition



- **'ack'** = an L1 **counting semaphore** (`tiles_acked`): the consumer **increments** it, the producer **polls** it.
 - **the race** : is that increment visible cross-core?"
- **We apply BOS's own BOS-ARCH-006 barrier** to the SDPA kernels BOS left out (macros `SDPA_HANG_WA` / `SDPA_DECODE_HANG_WA`).
 - `cb_pop_front<HANG_WA>` → runs a **tensix_sync()** full barrier → the pop is guaranteed **complete & visible**.
- **Effect** : no more hangs, 64k context completes.
- **Cost** : `tensix_sync()` = single-Tensix **full pipeline drain** → breaks unpack/math/pack overlap → **decode serialized**

Bug Fix

How BOS improves the SDPA hang fix : from SDK patch to Silicon

1

Extend our fix to the SDPA kernels

SDK patch

2

A scoped ack-fence, not a full-pipe drain

3

Order intra-Tensix L1 writes in HW / firmware

Silicon / firmware

- `tensix_sync()` **drains the whole Tensix pipeline** (unpack · math · pack → idle) on every ack. : heavy - ①
- **If a lighter fence holds** → decode can only stay equal or get faster - ②

BOS can adopt ① or ② or ③, from a quick patch to the root fix.

Conclusion

Conclusion

Goal & Achieved

64k decode 5.12 → 10.83 tok/s (2.1×)

Goal

- Accelerate 64k long-context decode on Eagle-N
- Beat the 5.12 tok/s baseline –while preserving accuracy

Achieved

- **Kernel optimization** : 5.12 → **8.20 tok/s (+60%)**, PCC>0.999
- **Quest** : 8.20 tok/s → **10.83 tok/s (+32%)**
- **Accuracy preserved** : 94% of the full-KV baseline while reading only 12.5% of the KV
- **SDPA hang fixed** : 64k runs at all — a bug BOS missed

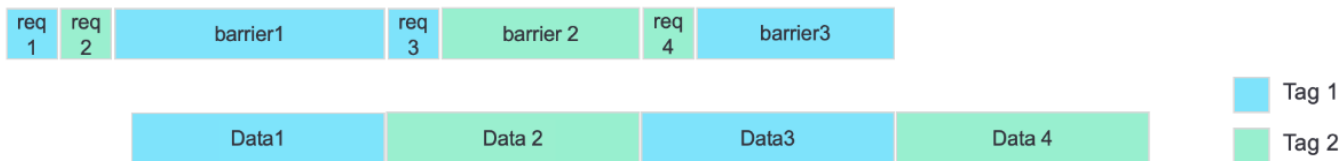
- END -

Questions or comments?

Appendix

Appendix A. Kernel Optimization Rationale

DRAM μ Benchmark – In-flight Requests

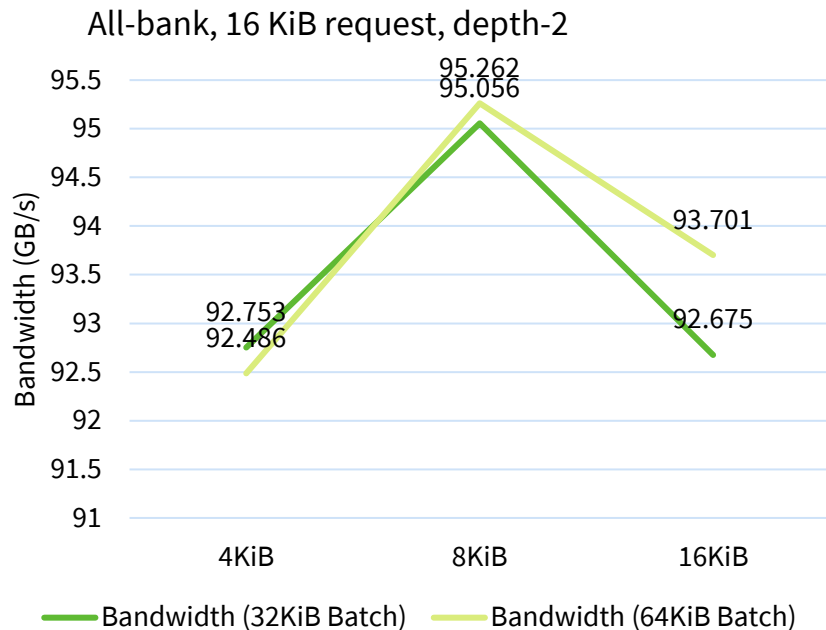
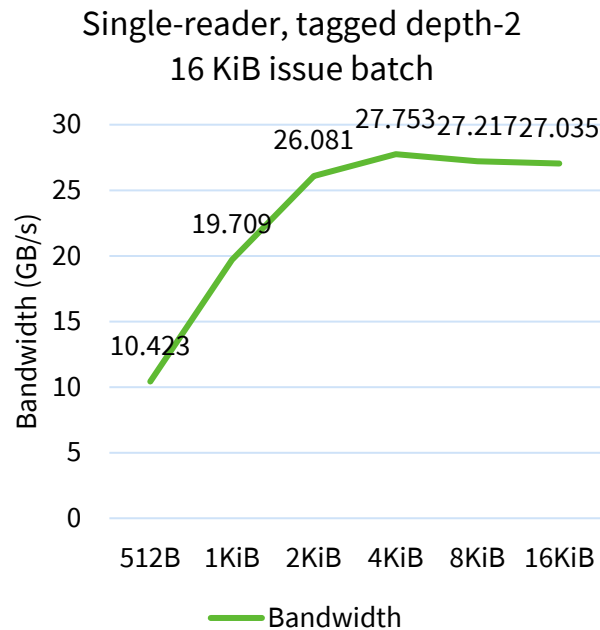


Mode	Bandwidth	vs. Depth-2
Tagged depth-2	95.792 GB/s	Baseline
Tagged depth-3	94.861 GB/s	-0.97%
Full barrier	67.673 GB/s	-29.35%

- To achieve bank saturation, at least one pending request is required
- Additional pending requests do not yield extra bandwidth.

Appendix A. Kernel Optimization Rationale

DRAM μ Benchmark – Request, Total batch sizes



- Single Bank – Single Reader Saturates at 2KiB/[DRAM NoC request]. **1kiB/request (1 BFP8 tile) or 512B/request(1 BFP4 tile) cannot saturate DRAM bandwidth.**
- Further batch size has only subtle impact on overall DRAM bandwidth

Appendix A. Kernel Optimization Rationale

Roofline Graph

BFP8 Tile Size

BFP8 Tile = 1088B

Blocked Floating-point

(0,0) 1B	...	(0,31)
...	...	...
(31,0)	...	(31,31)

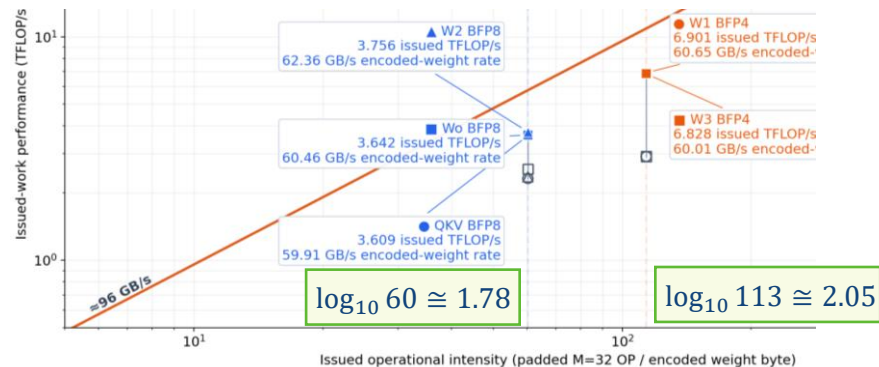
Block metadata (64B)

Matmul

SrcA (L1)
32x32 Tile

SrcB (DRAM)
32x32 Tile

Dst. (L1)
32x32 Tile



- Matmul operational intensity = $2(\text{multiply, add}) * 32(\text{Dim}) * 1024(\text{Total Elements}) / \text{DRAM Traffic}$
- For BFP8 SrcB: $65536 / 1088 = 60.235 \text{ OP/B}$ (KV cache and Wo, W2 weights are BFP8 format)
- For BFP4 SrcB: $65536 / 576 = 113.778 \text{ OP/B}$ (W1, W3 weights are BFP4 format)

Appendix A. Kernel Optimization Rationale

Scale Dot Product Attention

1. Balance Reader-Endpoint matching & Utilize all endpoints

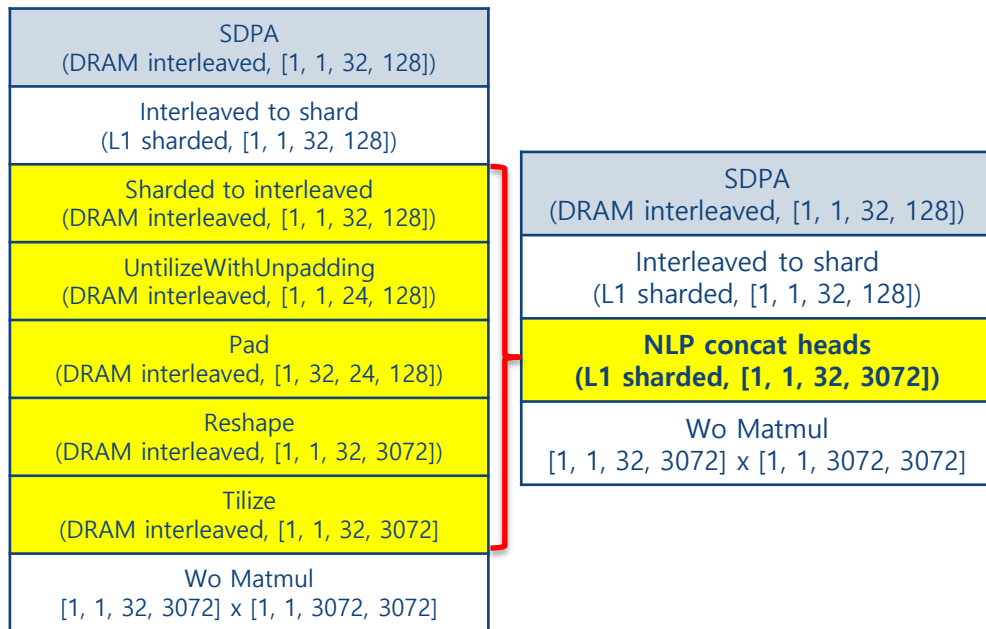
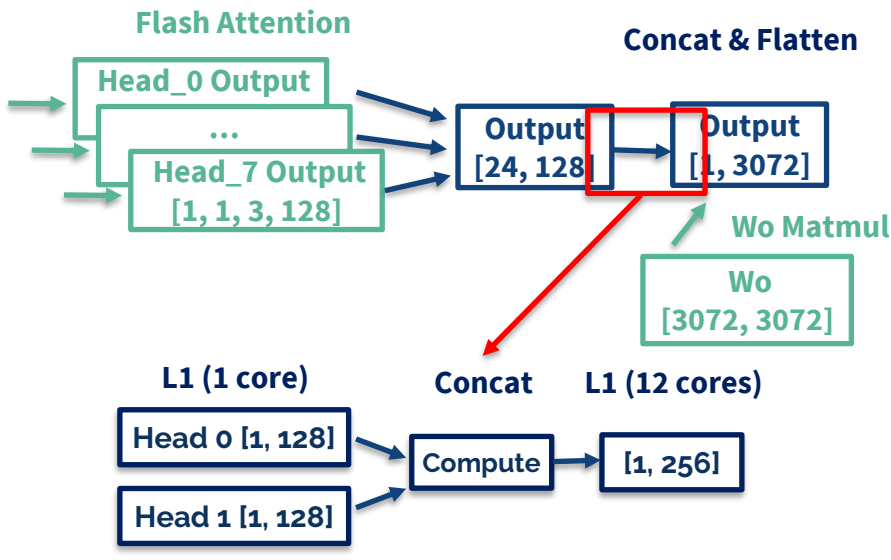
Metric	3 endpoints	6 endpoints	Effect
Endpoint load	5/0/5/6/0/0	3/3/2/3/3/2	Utilize full six DRAM endpoints
Profiled critical span	3.014568 ms	2.408938 ms	-20.09%
Effective K/V bandwidth	47.306 GB/s	59.199 GB/s	+25.14%
Reader K+V barrier mean	1669.096 us	556.299 us	-66.67%
Compute K+V input wait mean	396.584 us	22.817 us	-94.25%

- Utilizing 6 endpoints makes Bandwidth utilization better.
- It solely reduce kernel duration by 20%.

Appendix A. Kernel Optimization Rationale

Scale Dot Product Attention

2. Fuse concatenation operation



- Fuse the generic concat and layout-conversion pipeline into a single L1-to-L1 data-movement operation.
- Each of the 12 cores concatenates two attention heads directly into a $[32, 256]$ output shard.
- The Wo matmul kernel directly consumes the L1-sharded flattened tensor, avoiding intermediate DRAM materialization.
- Reducing the kernel duration **from 100 μ s to 5 μ s improved the overall throughput by 3%**

Appendix A. Kernel Optimization Rationale

SDPA

3. Double chunk size to reduce Fixed Cost

K chunk	Fully-optimized SDPA time	Changes	DRAM BW
128 tokens	2.51933 ms	Baseline	56.605 GB/s
256 tokens	2.03641 ms	-19.17%	70.028 GB/s
512 tokens	2.00242 ms	-20.52%	71.217 GB/s

Phase	Vanilla K128	Vanilla K256	K256 - K128
QK	550.922 us	917.908 us	+366.986 us
PV	617.582 us	739.639 us	+122.057 us
Current softmax	1014.236 us	794.670 us	-219.566 us
Online merge	844.290 us	424.637 us	-419.653 us
Total	3027.030 us	2876.854 us	-150.176 us

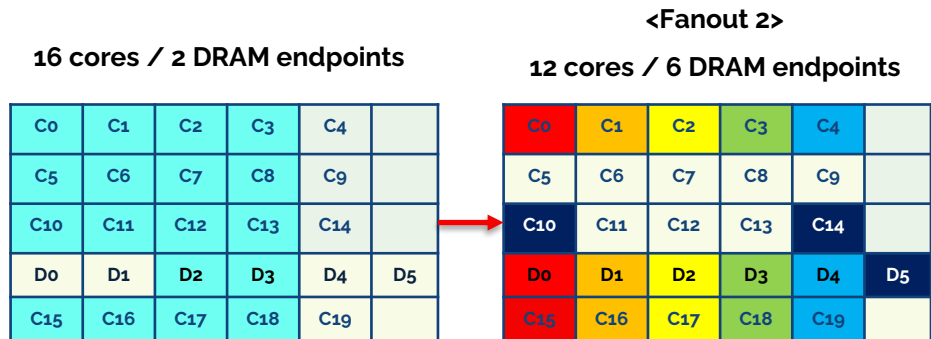
Static work item	K128	K256	Changes
Chunks per active core	256	128	-50%
QK calls	256	128	-50%
Current-softmax calls	256	128	-50%
PV calls	256	128	-50%
Online merges	255	127	-50.20%
K+V payload	136 MiB	136 MiB	-

- Increasing KV chunk to 256 raise QK, PV because awaiting big worksets, but reduce online Softmax/Merge
- K256 reduces SDPA kernel latency by **19%** compared with K128, with both configurations fully optimized.

Appendix A. Kernel Optimization Rationale

FeedForward (w/ QKV, Wo)

1. Balance Reader-Endpoint matching & Utilize all endpoints



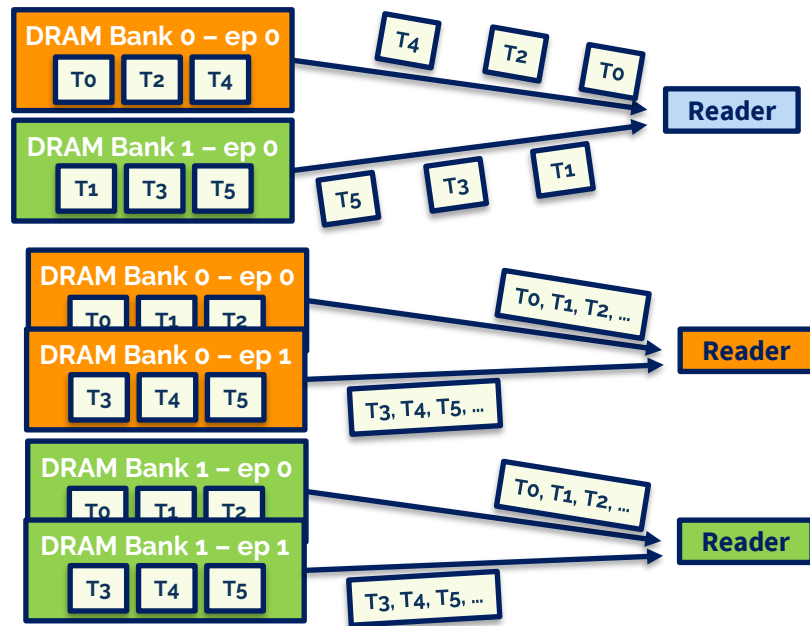
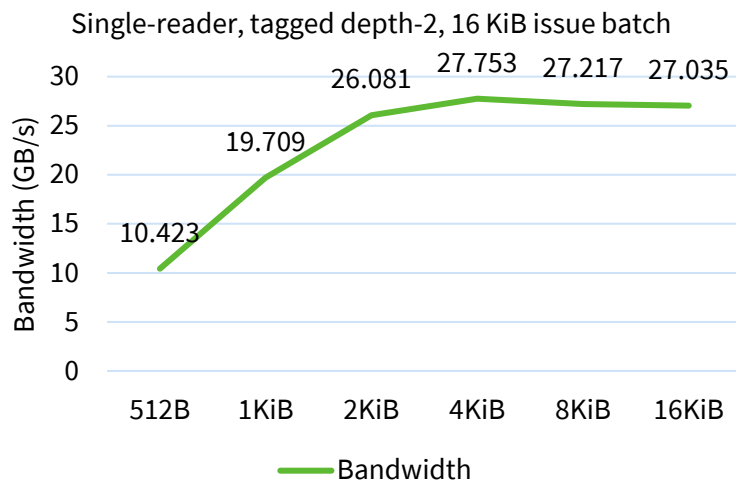
Config	Reader/c ompute	DRAM mapping	Mean latency
Baseline interleaved	16/16	2 physical endpoints, 8:8	1.974 ms
Fanout-2 interleaved	12/12	6 interface workers, NOC1 destination 4:4:4	1.606 ms

- Same color indicates cores streaming from the same DRAM-local shard.
- Each DRAM bank serves two shard-local endpoint groups.
- FeedForward with 12 compute / 6 endpoints reduces latency by 1.974 → 1.606 ms (-18.64%)

Appendix A. Kernel Optimization Rationale

FeedForward (w/ QKV, Wo)

2. DRAM width-sharded weight layout with contiguous shard-local reads

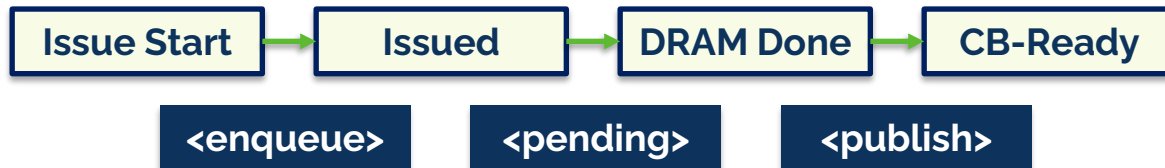


- DRAM sharding can coalesce adjacent tiles into larger contiguous requests, improving spatial locality and reducing request overhead.

Appendix A. Kernel Optimization Rationale

FeedForward (w/ QKV, Wo)

2. DRAM width-sharded weight layout with contiguous shard-local reads



projection	layout	<enqueue> start→issued (us)	<pending> issued→DRA M-done (us)	<publish> done→CB- ready (us)	Total (us)
W1 (BFP4)	width-sharded	3.294	42.345	0.080	24.272
W1 (BPF4)	interleaved	25.851	29.908	0.082	43.591
W2 (BFP8)	width-sharded	6.268	37.630	0.075	43.973
W2 (BFP8)	interleaved	25.001	29.176	0.077	54.253

- Width sharding cuts request-enqueue time by 98.4% for BFP4 W1/W3 and 74.9% for BFP8 W2, while reducing issue-to-CB-ready latency by about 44% and 19%, respectively.
- FeedForward with width-sharded & continuous shard local read reduces latency by 1.606 → 1.111 ms (-30.81%)

Appendix B.

Quest criticality

$$\text{criticality}(\text{page}) = \sum_d \max(Q_d \cdot \min_d, Q_d \cdot \max_d)$$

How the score is computed

- **Min-Key / Max-Key** : per page: the element-wise min & max over all its Keys (2 summary vectors, built once at prefill).
- **max(Q·min , Q·max) per dim** : the bigger product ($Q>0 \rightarrow Q \cdot \max$, $Q<0 \rightarrow Q \cdot \min$) = the biggest score this dimension can add.
- **Σ over dims** : the highest Q·Key any Key in the page could reach = the page's **criticality** (from 2 vectors, not every Key).
- **each decode step** : score all pages $\rightarrow$ keep **top-K** $\rightarrow$ attention on those pages only (sparse; full KV kept).